Deployment Plan: Two-Tier Flask & MySQL Application

Company: Modern India Pvt Limited

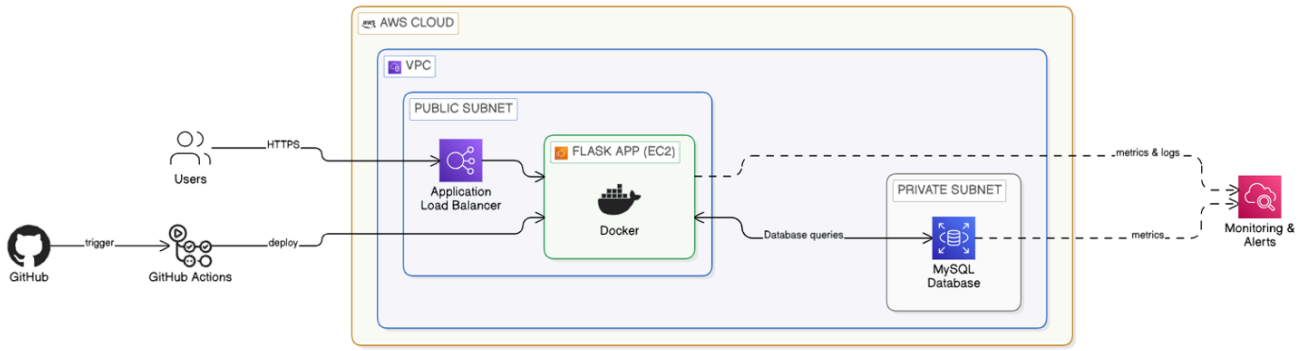
Prepared By: Vraj Lalwala, Technical Intern

Date: March 5, 2026

1. **Executive Summary**

This project outlines the strategy for deploying a containerized Flask application and a MySQL database on AWS. The goal is to move from a GitHub repository to a highly available, secure, and monitored production environment using industry-standard DevOps practices.

To visualize the production-grade deployment of the Two-Tier Flask & MySQL application, the following AWS architecture diagram has been designed. This architecture prioritizes security, high availability, and separation of concerns.



---

2. **Architecture Overview (The "What")**

The application follows a Two-Tier Architecture:

- Tier 1 (Web Tier): Flask application hosted on AWS EC2 (within a Docker container).
- Tier 2 (Data Tier): Amazon RDS (MySQL) for managed database services.
- Networking: A custom VPC with Public subnets (for the Web) and Private subnets (for the Database).

---

3. **Design Decisions (The "Why")**

Component	Choice	Why?
Compute	AWS EC2 (Dockerized)	Provides full control over the environment while ensuring portability via Docker containers.
Database	Amazon RDS (MySQL)	Offloads patching, backups, and scaling to AWS. Using a managed service is a "best practice" for reliability.
Networking	VPC with Private Subnet	Placing the database in a private subnet ensures it is not accessible from the open internet, reducing the attack surface.
CI/CD	GitHub Actions	Direct integration with the source code repo for automated testing and deployment.
Monitoring	CloudWatch & Grafana	CloudWatch provides native AWS metrics, while Grafana offers professional-grade visualization for the stakeholders.
Security	ACM & Security Groups	SSL/TLS (ACM) encrypts data in transit, and Security Groups act as a virtual firewall (Principle of Least Privilege).

---

4. **Implementation Strategy (The "How")**

Phase 1: Infrastructure Setup (Networking)

1. VPC Creation: Create a VPC (e.g., 10.0.0.0/16).

2. Subnets: Create two Public Subnets (Web) and two Private Subnets (DB) across different Availability Zones (AZs).
3. IGW: Attach an Internet Gateway to allow web traffic.
4. Security Groups:
   - Web SG: Allow Port 80 (HTTP), 443 (HTTPS), and 22 (SSH - restricted to your IP).
   - DB SG: Allow Port 3306 only from the Web SG.

Phase 2: Database Layer

1. Provision an Amazon RDS MySQL instance.
2. Select "Multi-AZ" for high availability.
3. Store database credentials securely (using AWS Secrets Manager or Environment Variables).

Phase 3: Application & Containerization

1. Dockerize: Create a Dockerfile for the Flask app and a docker-compose.yml to orchestrate the local environment.
2. Push Image: Build the image and push it to Amazon ECR (Elastic Container Registry).
3. Deploy: Pull the image onto the EC2 instance and run it.

Phase 4: Monitoring & Observability

1. CloudWatch: Enable detailed monitoring for EC2 (CPU/Memory) and RDS (Connections/Storage).
2. Uptime Kuma or CloudWatch Synthetics: To monitor the URL endpoint 24/7.
3. Grafana: Connect CloudWatch as a data source to create a "NOC-style" dashboard.

---

5. **Security & Best Practices**

- SSL/TLS: Use AWS Certificate Manager (ACM) with an Application Load Balancer (ALB) to enforce HTTPS.
- No Hardcoding: Ensure no MySQL passwords exist in the GitHub code. Use .env files or AWS Secrets Manager.
- IAM Roles: Use IAM Roles for EC2 instead of storing access keys on the server.